

HDOC 27

Name: Keshav Mittal

SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start main program.
2. Display the menu with options 1 to 6 to the user.
3. Read the user's choice.
4. Use a switch statement to call the respective function or exit if choice is 6.
5. Function fullPyramid(): Read n. Loop i from 1 to n. Inside, loop j from 1 to n - i printing spaces. Loop k from 1 to $2 * i - 1$ printing "* ". Print newline.
6. Function invertedPyramid(): Read n. Loop i from n down to 1. Inside, loop j from 1 to n - i printing spaces. Loop k from 1 to $2 * i - 1$ printing "* ". Print newline.
7. Function diamondPattern(): Read n. Execute the logic of fullPyramid() for the top half (rows 1 to n). Then, execute the logic of invertedPyramid() for the bottom half starting from row n - 1 down to 1.
8. Function palindromicPyramid(): Read n. Loop i from 1 to n. Loop j from 1 to n - i printing spaces. Loop j from 1 to i printing j. Loop j from i - 1 down to 1 printing j. Print newline.
9. Function repeatedNumberPyramid(): Read n. Loop i from 1 to n. Loop j from 1 to n - i printing spaces. Loop k from 1 to $2 * i - 1$ printing i. Print newline.
10. Repeat steps 2-4 until choice == 6. Stop.

2. Test-Case Table

TC ID	Scenario Description	Inputs	Expected Output
TC-01	Full pyramid (Valid)	choice: 1, n: 4	4 rows, centered, bottom row has 7 asterisks
TC-02	Inverted pyramid (Valid)	choice: 2, n: 4	4 rows, centered, top row has 7 asterisks
TC-03	Diamond pattern (Valid)	choice: 3, n: 4	7 rows total, middle row has 7 asterisks
TC-04	Palindromic number pyramid	choice: 4, n: 4	Row 4 is "1 2 3 4 3 2 1" centered
TC-05	Repeated-number pyramid	choice: 5, n: 4	Row 4 is "4 4 4 4 4 4 4" centered
TC-06	Invalid dimension (Boundary)	choice: 1, n: 0	Error message: Positive integer required.
TC-07	Invalid menu choice	choice: 7	Invalid choice. Please select 1-6.

3. C Program Source Code

```
#include <stdio.h>

// Function Prototypes void
fullPyramid(void);
void invertedPyramid(void); void
diamondPattern(void); void
palindromicPyramid(void);
void repeatedNumberPyramid(void);

int main() {
    int choice; do {
        printf("\n--- Pyramid & Diamond Patterns Menu ---\n"); printf("1. Full pyramid\n");
        printf("2. Inverted pyramid\n"); printf("3. Diamond
        pattern\n");
        printf("4. Palindromic number pyramid\n"); printf("5. Repeated-
        number pyramid\n"); printf("6. Exit\n");
        printf("Enter your choice: "); scanf("%d", &choice);

        switch (choice) {
            case 1: fullPyramid(); break; case 2:
            invertedPyramid(); break; case 3:
            diamondPattern(); break;
            case 4: palindromicPyramid(); break; case 5:
            repeatedNumberPyramid(); break;
            case 6: printf("Exiting program...\n"); break;
            default: printf("Error: Invalid choice. Please enter 1-6.\n");
        }
    } while (choice != 6);

    return 0;
}

void fullPyramid(void) { int n, i, j, k;
    printf("Enter number of rows n: "); scanf("%d", &n);
    if (n <= 0) { printf("Error: Positive integer required.\n"); return; }

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("* "); printf("\n");
    }
}

void invertedPyramid(void) { int n, i, j, k;
    printf("Enter number of rows n: "); scanf("%d", &n);
    if (n <= 0) { printf("Error: Positive integer required.\n"); return; }
```

```

    for (i = n; i >= 1; i--) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("* "); printf("\n");
    }
}

void diamondPattern(void) { int n, i, j, k;
    printf("Enter number of rows for upper half n: "); scanf("%d", &n);
    if (n <= 0) { printf("Error: Positive integer required.\n"); return; }

    // Upper half including middle for (i = 1; i <= n;
    i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("* "); printf("\n");
    }
    // Lower half
    for (i = n - 1; i >= 1; i--) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("* "); printf("\n");
    }
}

void palindromicPyramid(void) { int n, i, j;
    printf("Enter number of rows n: "); scanf("%d", &n);
    if (n <= 0) { printf("Error: Positive integer required.\n"); return; }

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= i; j++) printf("%d ", j);
        for (j = i - 1; j >= 1; j--) printf("%d ", j); printf("\n");
    }
}

void repeatedNumberPyramid(void) { int n, i, j, k;
    printf("Enter number of rows n: "); scanf("%d", &n);
    if (n <= 0) { printf("Error: Positive integer required.\n"); return; }

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("%d ", i); printf("\n");
    }
}

```

4. Compilation and Execution

```
(kali@kali)-[~/liak]
$ cc hdoc27.c

(kali@kali)-[~/liak]
$ ./a.out

— Pyramid & Diamond Patterns Menu —
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 2
Enter number of rows n: 4
* * * * *
 * * * *
  * * *
   * *
    *

— Pyramid & Diamond Patterns Menu —
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 5
Enter number of rows n: 3
  1
 2 2 2
3 3 3 3 3

— Pyramid & Diamond Patterns Menu —
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 6
Exiting program...

(kali@kali)-[~/liak]
$
```